STN-7 Problem Solving Assessment List

General Instructions

1. Solve all 20 assigned problems in JavaScript or TypeScript.
2. Implement all solutions in **one single file**. Do not create separate files for individual problems.
3. Use the exact function names and parameter structures provided in each problem.
4. Do not use hardcoded outputs. Your solutions must work with different valid inputs.
5. Consider edge cases, input constraints, and time and space complexity.
6. For in-place problems, modify the input array as required instead of returning a newly created array when the function signature specifies void.
7. Test your solutions with the provided examples and additional test cases.
8. Organize your code using comments to identify each problem clearly.

Problem 1: Staircase

Problem Description

You are given an integer n, representing the height of a staircase. Your task is to construct a right-aligned staircase using the # character and spaces.

The staircase must contain exactly n rows. Each row should contain the appropriate number of leading spaces followed by # characters. The first row contains one #, and each subsequent row contains one additional # until the final row contains n characters.

Each row must be separated by a newline character (\n). Do not include unnecessary spaces at the end of any row.

Function Signature

```
/**
 * @param {number} n
 * @return {string}
 */
var staircase = function(n) {

};
```

Examples

Example 1

Input:

n = 4

Output:

```
   #
  ##
 ###
####
```

Example 2

Input:

n = 1

Output:

#

Problem 2: Grading Students

Problem Description

A teacher has assigned grades to a group of students. The grading system allows grades to be rounded according to the following rules:

1. If a grade is less than 38, it remains unchanged because it is below the minimum passing grade.
2. If the difference between the grade and the next multiple of 5 is less than 3, round the grade up to that next multiple of 5.
3. Otherwise, keep the original grade unchanged.

Given an array of student grades, return a new array containing the final grades after applying these rules independently to each student.

Function Signature

```
/**
 * @param {number[]} grades
 * @return {number[]}
 */
var gradingStudents = function(grades) {

};
```

Examples

Example 1

Input:

grades = [73, 67, 38, 33]

Output:

[75, 67, 40, 33]

Explanation:

- 73 is rounded to 75 because the difference is 2.
- 67 remains unchanged because the difference to 70 is 3.
- 38 is rounded to 40 because the difference is 2.
- 33 remains unchanged because it is below 38.

Example 2

Input:

grades = [84, 29, 57]

Output:

[85, 29, 57]

Problem 3: Time Conversion

Problem Description

You are given a time string in 12-hour clock format, including hours, minutes, seconds, and an AM/PM suffix.

Convert the given time into its equivalent 24-hour clock format.

The 24-hour format uses hours from 00 through 23 and does not include an AM/PM suffix.

Remember the following special cases:

- 12:xx:xxAM becomes 00:xx:xx.
- 12:xx:xxPM remains 12:xx:xx.
- Any other PM hour must be increased by 12.
- Other AM hours remain unchanged.

Return the converted time as a string in HH:MM:SS format.

Function Signature

```
/**
 * @param {string} s
 * @return {string}
 */
var timeConversion = function(s) {

};
```

Examples

Example 1

Input:

s = "07:05:45PM"

Output:

"19:05:45"

Example 2

Input:

s = "12:00:00AM"

Output:

"00:00:00"

Example 3

Input:

s = "12:30:45PM"

Output:

"12:30:45"

Problem 4: String Repeat

Problem Description

Given a non-negative integer n and a string s, return a new string formed by repeating s exactly n times.

The resulting string must preserve the original order of the characters in s. If n is zero, return an empty string.

You may use built-in string methods or implement the repetition using a loop.

Function Signature

```
/**
 * @param {number} n
 * @param {string} s
 * @return {string}
 */
var repeatStr = function(n, s) {

};
```

Examples

Example 1

Input:

n = 3, s = "ha"

Output:

"hahaha"

Example 2

Input:

n = 0, s = "hello"

Output:

""

Example 3

Input:

n = 4, s = "a"

Output:

"aaaa"

Problem 5: Reversed Strings

Problem Description

Given a string str, return a new string containing the same characters in reverse order.

The first character of the original string must become the last character of the result, and the last character of the original string must become the first character.

The original string must not be modified. An empty string should return an empty string.

Function Signature

```
/**
 * @param {string} str
 * @return {string}
 */
var solution = function(str) {

};
```

Examples

Example 1

Input:

str = "world"

Output:

"dlrow"

Example 2

Input:

str = "hello"

Output:

"olleh"

Example 3

Input:

str = ""

Output:

""

Problem 6: Sum of Positive

Problem Description

Given an array of integers, calculate the sum of all positive numbers in the array.

A positive number is any number strictly greater than zero. Ignore zero and all negative numbers.

If the array contains no positive numbers or is empty, return 0.

Function Signature

```
/**
 * @param {number[]} arr
 * @return {number}
 */
var positiveSum = function(arr) {

};
```

Examples

Example 1

Input:

arr = [1, -4, 7, 12]

Output:

20

Explanation: The positive numbers are 1, 7, and 12. Their sum is 20.

Example 2

Input:

arr = [-1, -2, 0, -5]

Output:

0

Problem 7: Grasshopper — Summation

Problem Description

Given a positive integer num, calculate the sum of all integers from 1 through num, inclusive.

For example, when num = 4, the required sum is:

[
1 + 2 + 3 + 4 = 10
]

Return the calculated sum as an integer.

Consider an iterative solution and the mathematical formula for the sum of consecutive positive integers.

Function Signature

```
/**
 * @param {number} num
 * @return {number}
 */
var summation = function(num) {

};
```

Examples

Example 1

Input:

num = 8

Output:

36

Example 2

Input:

num = 1

Output:

1

Problem 8: Convert a Boolean to a String

Problem Description

Given a Boolean value b, convert it into its corresponding string representation.

If b is true, return the string "true". If b is false, return the string "false".

The return value must be a string, not a Boolean.

Function Signature

```
/**
 * @param {boolean} b
 * @return {string}
 */
var booleanToString = function(b) {

};
```

Examples

Example 1

Input:

b = true

Output:

"true"

Example 2

Input:

b = false

Output:

"false"

Problem 9: Fizz Buzz

Problem Description

Given an integer `n`, generate a sequence of strings representing the integers from `1` through `n`.

For each integer `i`, apply the following rules in order:

- If `i` is divisible by both `3` and `5`, use `"FizzBuzz"`.
- If `i` is divisible by `3` only, use `"Fizz"`.
- If `i` is divisible by `5` only, use `"Buzz"`.
- Otherwise, use the decimal string representation of `i`.

Return an array containing exactly `n` strings in the same order as the integers.

Important: Check divisibility by both `3` and `5` before checking either one individually.

Function Signature

```
/**
 * @param {number} n
 * @return {string[]}
 */
var fizzBuzz = function(n) {

};
```

Examples

Example 1

Input:

n = 5

Output:

["1", "2", "Fizz", "4", "Buzz"]

Example 2

Input:

n = 15

Output:

["1", "2", "Fizz", "4", "Buzz", "Fizz", "7", "8", "Fizz", "Buzz", "11", "Fizz", "13", "14", "FizzBuzz"]

Problem 10: Running Sum of 1D Array

Problem Description

Given an array of integers nums, calculate its running sum.

The running sum at index i is the sum of all elements from index 0 through index i, inclusive.

Formally:

[
runningSum[i] = \sum_{j=0}^{i} nums[j]
]

Return a new array containing the running sum at every index.

Function Signature

```
/**
 * @param {number[]} nums
 * @return {number[]}
 */
var runningSum = function(nums) {

};
```

Examples

Example 1

Input:

nums = [1, 2, 3, 4]

Output:

[1, 3, 6, 10]

Explanation:

- Index 0: 1
- Index 1: 1 + 2 = 3
- Index 2: 1 + 2 + 3 = 6
- Index 3: 1 + 2 + 3 + 4 = 10

Example 2

Input:

nums = [1, 1, 1, 1, 1]

Output:

[1, 2, 3, 4, 5]

Problem 11: Richest Customer Wealth

Problem Description

You are given a two-dimensional array called accounts, where each row represents a customer and each element in that row represents the amount of money the customer holds in a particular bank account.

The total wealth of a customer is the sum of all values in their row.

Your task is to calculate the total wealth of every customer and return the maximum total wealth found.

Assume that all customers have the same number of bank accounts.

Function Signature

```
/**
 * @param {number[][]} accounts
 * @return {number}
 */
var maximumWealth = function(accounts) {

};
```

Examples

Example 1

Input:

accounts = [[1, 2, 3], [3, 2, 1]]

Output:

6

Explanation: Both customers have a total wealth of 6.

Example 2

Input:

accounts = [[1, 5], [7, 3], [3, 5]]

Output:

10

Explanation: The second customer has the highest wealth: 7 + 3 = 10.

Problem 12: Remove Duplicates from Sorted Array

Problem Description

Given an integer array nums sorted in non-decreasing order, remove duplicate values in-place so that each unique value appears exactly once.

The relative order of the unique elements must remain unchanged.

After removing duplicates, return the number of unique elements, k. The first k positions of nums must contain the unique elements in sorted order.

You must modify the original array without creating another array to store the unique values.

The contents of nums beyond index k - 1 do not matter.

Function Signature

```
/**
 * @param {number[]} nums
 * @return {number}
 */
```

```
var removeDuplicates = function(nums) {

};
```

Examples

Example 1

Input:

nums = [1, 1, 2]

Output:

2

After the function executes, the first two elements of nums must be:

[1, 2]

Example 2

Input:

nums = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]

Output:

5

The first five elements must be:

[0, 1, 2, 3, 4]

Problem 13: Contains Duplicate

Problem Description

Given an integer array nums, determine whether any value appears at least twice.

Return true if the array contains one or more duplicate values. Return false if every element is distinct.

Two elements are considered duplicates if they have the same integer value, regardless of their positions in the array.

Function Signature

```
/**
 * @param {number[]} nums
 * @return {boolean}
 */
var containsDuplicate = function(nums) {

};
```

Examples

Example 1

Input:

nums = [1, 2, 3, 1]

Output:

true

Explanation: The value 1 appears twice.

Example 2

Input:

nums = [1, 2, 3, 4]

Output:

false

Explanation: All elements are distinct.

Example 3

Input:

nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]

Output:

true

Problem 14: Move Zeroes

Problem Description

Given an integer array nums, move all occurrences of zero to the end of the array while maintaining the relative order of all non-zero elements.

You must perform this operation in-place, without creating another array for the result.

For example, if the input is [0, 1, 0, 3, 12], the non-zero elements must remain in the order 1, 3, 12, and all zeros must be placed after them.

Your function should modify the original array and return nothing.

Function Signature

```
/**
 * @param {number[]} nums
 * @return {void}
 */
var moveZeroes = function(nums) {

};
```

Examples

Example 1

Input:

nums = [0, 1, 0, 3, 12]

Output after modification:

[1, 3, 12, 0, 0]

Example 2

Input:

nums = [0, 0, 1]

Output after modification:

[1, 0, 0]

Example 3

Input:

nums = [1, 2, 3]

Output after modification:

[1, 2, 3]

Problem 15: Merge Sorted Array

Problem Description

You are given two integer arrays, nums1 and nums2, sorted in non-decreasing order, along with two integers m and n.

- nums1 has a total length of m + n. Its first m positions contain valid elements, while the remaining n positions are initialized to zero and reserved for the merged result.
- nums2 contains exactly n valid elements.

Merge the elements of nums2 into nums1 so that the final array contains all m + n elements in non-decreasing order.

The merge must be performed directly in nums1. Do not return a separate merged array.

Important: The zeros reserved at the end of nums1 are empty positions, not necessarily actual input values.

Function Signature

```
/**
 * @param {number[]} nums1
 * @param {number} m
 * @param {number[]} nums2
 * @param {number} n
 * @return {void}
 */
var merge = function(nums1, m, nums2, n) {

};
```

Examples

Example 1

Input:

nums1 = [1, 2, 3, 0, 0, 0]
m = 3

nums2 = [2, 5, 6]
n = 3

Output after modification:

[1, 2, 2, 3, 5, 6]

Example 2

Input:

nums1 = [1]
m = 1
nums2 = []
n = 0

Output after modification:

[1]

Example 3

Input:

nums1 = [0]
m = 0
nums2 = [1]
n = 1

Output after modification:

[1]

Problem 16: 3Sum

Problem Description

Given an integer array nums, find all unique triplets [nums[i], nums[j], nums[k]] such that the indices are distinct and the sum of the three corresponding values is zero.

Formally:

[
nums[i] + nums[j] + nums[k] = 0
]

Your solution must satisfy the following requirements:

- Each triplet must contain three elements whose sum is exactly zero.
- The same array element cannot be used more than once in a single triplet.
- The result must not contain duplicate triplets, even if the input contains repeated values.
- The order of the triplets in the final result does not matter.

Return an array containing all valid unique triplets. If no valid triplets exist, return an empty array.

Function Signature

```
/**
 * @param {number[]} nums
 * @return {number[][]}
 */
var threeSum = function(nums) {

};
```

Examples

Example 1

Input:

nums = [-1, 0, 1, 2, -1, -4]

Output:

[[-1, -1, 2], [-1, 0, 1]]

Explanation:

- -1 + -1 + 2 = 0
- -1 + 0 + 1 = 0

No other unique triplets satisfy the condition.

Example 2

Input:

nums = [0, 1, 1]

Output:

[]

Example 3

Input:

nums = [0, 0, 0]

Output:

[[0, 0, 0]]

Problem 17: Plus One

Problem Description

You are given a non-empty array of digits representing a non-negative integer.

Each element contains a single digit, and the digits are arranged from the most significant digit to the least significant digit. The integer does not contain leading zeros, except when the number itself is zero.

Increment the represented integer by one and return the resulting array of digits.

You must correctly handle carry propagation. For example, adding one to a number ending in 9 may require carrying into the preceding digit. If every digit is 9, the resulting array will contain one additional digit.

Function Signature

```
/**
 * @param {number[]} digits
 * @return {number[]}
 */
var plusOne = function(digits) {

};
```

Examples

Example 1

Input:

digits = [1, 2, 9]

Output:

[1, 3, 0]

Explanation: The number 129 becomes 130.

Example 2

Input:

digits = [4, 3, 2, 1]

Output:

[4, 3, 2, 2]

Example 3

Input:

digits = [9, 9, 9]

Output:

[1, 0, 0, 0]

Problem 18: Reverse String

Problem Description

You are given an array of characters s. Reverse the order of the characters in the array.

The reversal must be performed in-place, meaning you must modify the original array without creating another array to hold the reversed characters.

The function should return nothing. After the function executes, the original array must contain the characters in reverse order.

Consider using two pointers: one starting at the beginning of the array and another at the end.

Function Signature

```
/**
 * @param {string[]} s
 * @return {void}
 */
var reverseString = function(s) {

};
```

Examples

Example 1

Input:

s = ["h", "e", "l", "l", "o"]

Output after modification:

["o", "l", "l", "e", "h"]

Example 2

Input:

s = ["H", "a", "n", "n", "a", "h"]

Output after modification:

["h", "a", "n", "n", "a", "H"]

Problem 19: Length of Last Word

Problem Description

Given a string s consisting of words and spaces, return the length of the last word in the string.

A word is defined as a maximal sequence of non-space characters. Words are separated by one or more spaces.

The string may contain leading spaces, trailing spaces, or multiple consecutive spaces between words. These spaces must not affect the length of the last word.

Your function should return the number of characters in the final word, excluding any surrounding spaces.

Function Signature

```
/**
 * @param {string} s
 * @return {number}
 */
var lengthOfLastWord = function(s) {

};
```

Examples

Example 1

Input:

s = "Hello World"

Output:

5

Explanation: The last word is "World", which contains five characters.

Example 2

Input:

s = " fly me to the moon "

Output:

4

Explanation: The last word is "moon".

Example 3

Input:

s = "luffy is still joyboy"

Output:

6

Problem 20: Longest Common Prefix

Problem Description

Given an array of strings strs, find the longest common prefix shared by every string in the array.

A prefix is a sequence of characters that appears at the beginning of a string. The common prefix must match across all strings, character by character, starting from index zero.

If the strings do not share any common starting characters, return an empty string.

If the array contains only one string, that entire string is the longest common prefix. If any string is empty, the longest common prefix must also be empty.

Function Signature

```
/**
 * @param {string[]} strs
 * @return {string}
 */
var longestCommonPrefix = function(strs) {

};
```

Examples

Example 1

Input:

strs = ["flower", "flow", "flight"]

Output:

"fl"

Explanation: All three strings begin with "fl", but their next characters differ.

Example 2

Input:

strs = ["dog", "racecar", "car"]

Output:

""

Explanation: There is no common prefix shared by all three strings.

Example 3

Input:

strs = ["interspecies", "interstellar", "interstate"]

Output:

"inter"